



UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

Facultad de Ingeniería · Escuela Profesional de Ingeniería en Informática y Sistemas

Análisis Comparativo de Almacenamiento

HDD vs **SSD** vs **NVMe** — y Simulador de Rendimiento

Simulador propio en C++ con interfaz web (HTML) que estima y compara tiempos de transferencia de datos entre tecnologías de almacenamiento.

Angel Ccopa Escobar	2022-119003
Stip Leonel Yordy Corimaya Chura	2024-119032
Ronald Choquehuanca Rafael	2024-119019
Jhosep Rodrigo Arocutipa Mamani	2024-119027

Arquitectura de Computadores · Docente: Genyfer Margaret Aldana Salgado · Ciclo 5to · Tacna, Perú 2026



INTRODUCCIÓN

De la medición real a un motor de simulación propio

Versión inicial

Medición directa de discos reales con **CrystalDiskMark** y **CrystalDiskInfo**.

Versión actual

Programa propio en **C++** que calcula el tiempo de transferencia con valores estándar de fábrica, complementado con una interfaz **HTML** para interacción y visualización.

Cambia la metodología, no la meta: entender por qué HDD, SSD SATA y NVMe rinden distinto — y ahora esa lógica es responsabilidad directa del código del grupo.

OBJETIVOS

Qué busca demostrar el proyecto

OBJETIVO GENERAL

Elaborar y describir un simulador en C++ con interfaz HTML que calcule y contraste el tiempo de transferencia de un SSD NVMe, un SSD SATA y un HDD.

- Modelar el tiempo de transferencia secuencial de un archivo único de gran tamaño.
- Modelar la transferencia de archivos fragmentados, incorporando la latencia de acceso.
- Documentar y justificar las constantes de velocidad (MB/s) frente a especificaciones reales.
- Integrar el motor C++ con una interfaz HTML sin pasos intermedios de exportación.
- Explicar, desde la arquitectura, por qué difieren las constantes entre SATA/AHCI y PCIe/NVMe.

Tres tecnologías, tres arquitecturas

HDD

Disco duro mecánico

Platos giratorios (5400–7200 RPM) + cabezal móvil.

- Velocidad secuencial: 100–160 MB/s
- Seek time: varios ms por operación
- Menor costo por GB

SSD SATA

Estado sólido, interfaz SATA III

Memoria flash NAND, sin partes móviles.

- Límite teórico SATA III: ~600 MB/s
- Real: 450–550 MB/s
- Latencia: decenas–cientos de μ s

NVMe

Protocolo sobre bus PCIe

Elimina el cuello de botella de SATA.

- 3000–3500+ hasta 7000 MB/s (PCIe 3.0/4.0)
- Latencia del orden de μ s
- Colas masivamente paralelas

El simulador no mide: estima con constantes de referencia

150 MB/s

HDD (7200 RPM, SATA)

550 MB/s

SSD SATA III

7000 MB/s

SSD NVMe (PCIe)

Transferencia secuencial (archivo único)

$\text{tiempo} = \text{tamaño (MB)} \div \text{velocidad de referencia (MB/s)}$

Transferencia fragmentada (muchos archivos)

$\text{tiempo} = (\text{tamaño total} \div \text{velocidad}) + (\text{n}^\circ \text{ archivos} \times \text{latencia simulada})$

La latencia simulada es notablemente mayor para HDD (desplazamiento mecánico del cabezal) que para SSD o NVMe (acceso electrónico casi instantáneo).

El bus limita al dispositivo, no solo lo acompaña

SATA III / AHCI

- Ancho de banda teórico: ~6 Gb/s (≈600 MB/s efectivos)
- 1 sola cola de comandos
- Profundidad máxima: 32 operaciones
- 1 interrupción de CPU por comando

PCIe / NVMe

- Se ejecuta directamente sobre PCIe, sin controlador SATA
- Hasta 65 536 colas de comandos
- Hasta 65 536 operaciones por cola
- Procesamiento paralelo entre núcleos

Caso ilustrativo: un SSD NVMe en un bus PCIe 2.0 conserva el protocolo NVMe, pero su rendimiento queda acotado a ~500 MB/s por línea — muy por debajo de los 3500 MB/s usados como referencia para PCIe 3.0/4.0.

Estructura: C++ como motor, HTML como interfaz

.cpp

Lógica principal: menú interactivo, lectura de parámetros y funciones de cálculo.

.hpp

Declaraciones de funciones, estructuras y constantes de velocidad del simulador.

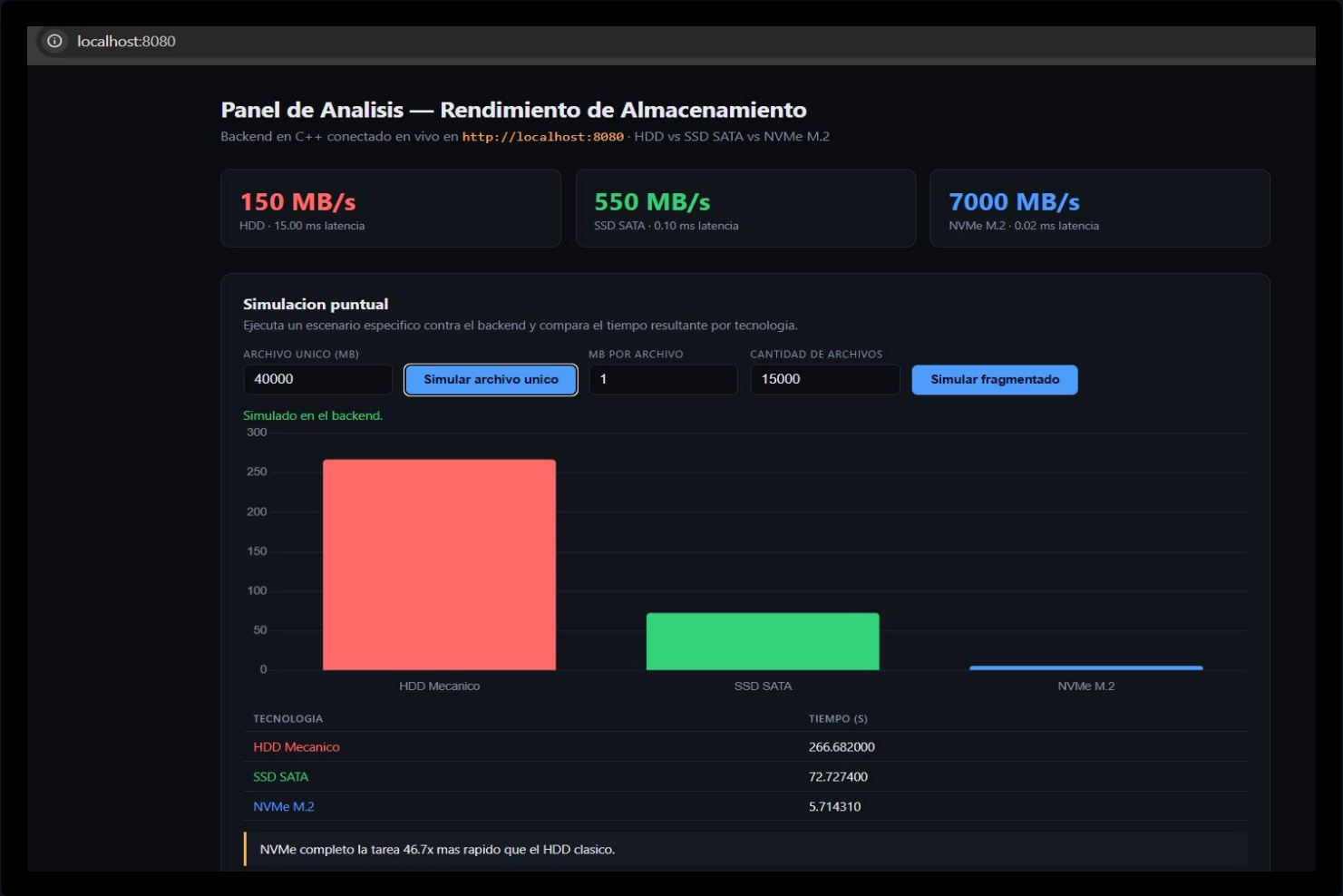
.html

Interfaz gráfica: ingreso de datos y visualización de resultados comparativos.

Organización en Visual Studio siguiendo la separación habitual de un programa C++ con interfaz web (backend, frontend, tests, web).

```
> backend
✓ frontend
  ↳ Colores.hpp
  ↳ ConsoleUI.cpp
  ↳ ConsoleUI.hpp
  ✓ tests
    ↳ test_disco.cpp
  ✓ web
    ↳ index.html
  M CMakeLists.txt
  ↳ main_servidor.cpp
  ↳ main.cpp
  M Makefile
  ⓘ README.md
  ≡ servidor_discos.exe
  ≡ simulador_discos.exe
  ≡ simulador.log
```

Opción 1 — Simular archivo único (transferencia secuencial)



El usuario ingresa el tamaño del archivo en MB; el backend en C++ calcula el tiempo de lectura/escritura para cada tecnología con:

$$t = \text{tamaño} \div \text{velocidad}$$

Resultado de ejemplo (40 000 MB):

- HDD Mecánico 266.68 s
- SSD SATA 72.73 s
- NVMe M.2 5.71 s

NVMe completa la tarea ≈ 46.7× más rápido que el HDD clásico.

ANÁLISIS COMPARATIVO

Latencia domina a la izquierda, ancho de banda a la derecha



Dos escenarios de simulación

Escenario 1 · Archivo único de 1024 MB (1 GB)

Tecnología	MB/s	Tiempo (s)
HDD	150	6.84
SSD SATA	550	1.86
NVMe	7000	0.14

Escenario 2 · 1000 archivos de 1 MB c/u (fragmentado)

Tecnología	Transf. (s)	Total (s)
HDD	6.666	23.83
SSD SATA	1.818	1.918
NVMe	0.143	0.163

NVMe vs. SSD SATA: ~12.7× más rápido en transferencia secuencial (7000 vs. 550 MB/s), por el mayor ancho de banda de PCIe frente a SATA III.

SSD SATA vs. HDD: ~3.7× más rápido en secuencial; la brecha crece en archivos fragmentados por la penalización mecánica del cabezal del HDD.

Opción 3 y 4 — Historial de sesión y ficha técnica

Historial de sesion

ActualizarExportar

ESCENARIO	TEC.	TIEMPO (S)
Archivo unico (40000 MB)	HDD Mecanico	266.682000
Archivo unico (40000 MB)	SSD SATA	72.727400
Archivo unico (40000 MB)	NVMe M.2	5.714310

Ficha tecnica

HDD Mecanico

150 MB/s · 15 ms latencia

Almacenamiento magnetico con partes moviles. La latencia es alta por el posicionamiento fisico del cabezal de lectura/escritura sobre el plato giratorio.

SSD SATA

550 MB/s · 0.1 ms latencia

Unidad de estado solido con acceso puramente electronico a compuertas NAND. Sin partes moviles, por lo que la latencia es baja y estable.

NVMe M.2

7000 MB/s · 0.02 ms latencia

Unidad de estado solido conectada directamente al bus PCIe del CPU, eliminando el cuello de botella del controlador SATA. Latencia ultra baja y maximo ancho de banda.

Todo el flujo — simular, comparar y consultar la teoría — ocurre dentro del mismo navegador, sin exportar archivos intermedios.

- Opción 1 Archivo único
- Opción 2 Fragmentados
- Opción 3 Ficha técnica
- Opción 4 Visualización

De consola desacoplada a experiencia integrada

01

Dependencia de archivos intermedios

C++ y HTML se comunicaban solo vía CSV / datos.js exportados manualmente tras cada corrida.

Solución: Simulación integrada directo en la interfaz web — sin pasos de exportación.

02

Validación de datos de entrada

Valores no numéricos, negativos o cero podían causar divisiones por cero.

Solución: Validar antes de calcular; rechazar y re-solicitar el dato.

03

Precisión en archivos muy pequeños

Pocos decimales redondeaban tiempos rápidos a 0.00 s.

Solución: Más decimales o unidades en milisegundos para mantener diferencias visibles.

CONCLUSIONES

Lo que demuestra el simulador

1

Se documentó un simulador C++/HTML que estima y compara HDD, SSD SATA y NVMe con constantes de referencia y latencia simulada.

2

El modelo reproduce las diferencias reales de la literatura: NVMe > SSD SATA > HDD, sobre todo con archivos pequeños y numerosos.

3

Las diferencias están fundamentadas en la arquitectura del bus y el protocolo (SATA/AHCI vs. PCIe/NVMe), no son arbitrarias.

4

La mejora clave fue integrar el motor C++ con la interfaz HTML, eliminando archivos intermedios (CSV, datos.js).

Recomendaciones futuras: validar constantes de latencia con datasheets reales · modo configurable por generación de bus (PCIe 2.0/3.0/4.0) · contrastar con mediciones reales de hardware.